

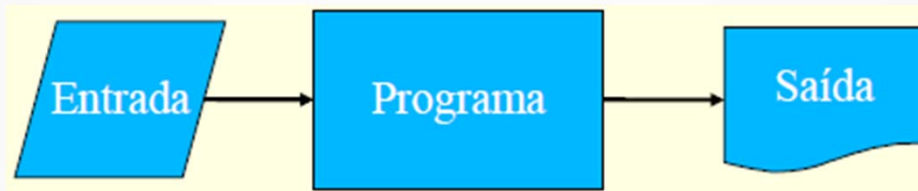
Introdução

Por que programar?

- É divertido
 - Melhor que usar programas feitos por outros
 - Fazer programas = resolver quebra-cabeças
 - Programação como arte
- É fácil
 - Programação como ferramenta
 - Pode ser aplicado a quase toda atividade
 - Arte / Ciência / Filosofia / Entretenimento

Algoritmos e Programas

- **Algoritmo** = método (conjunto de ferramentas) para solucionar um problema específico.
- **Estruturas de dados** = método para organizar dados.
- **Programa** = algoritmos + estruturas de dados expressos de forma a ser entendidos/executados pelo computador.
- Programas tipicamente processam dados de entrada e produzem dados de saída.



Lógica e Linguagem de Programação

- **Lógica de programação** é a organização coesa do pensamento em uma sequência de passos corretos para resolver um problema, ou criar um programa de computador, software ou aplicação.
- **Linguagem de programação** trata-se de linguagem artificial com um conjunto bem definido de regras (sintaxe e semântica), palavras e símbolos usado por pessoas para dar ordens ao computador.
 - Ex. Python, Pascal, C/C++, Java, PHP, Ruby, Dart, Lua, etc.
- Temos vários idiomas no mundo, também temos várias linguagens de programação que nos permitem “falar” com o computador.
 - Usada para exprimir programas.
 - Mais “técnica” do que linguagens naturais.
 - Menos “técnica” do que a linguagem nativa da máquina.

Compiladores

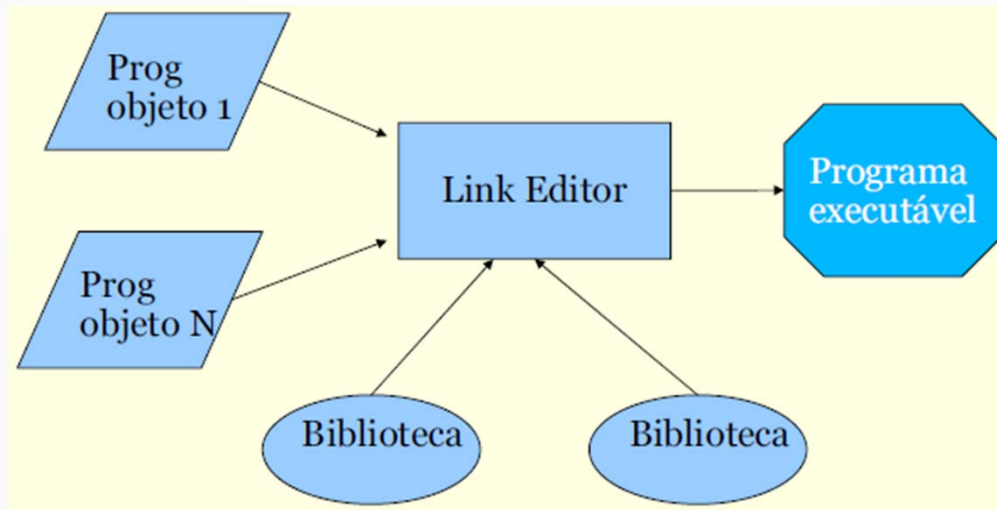
- **Compiladores** são programas que traduzem programas escritos em linguagem de programação para programas equivalentes escritos em linguagem de máquina.
 - Ex.: Pascal, C/C++ ...
- O primeiro é chamado de **programa-fonte**, enquanto que o segundo é chamado de **programa-objeto**.



Link-Editores

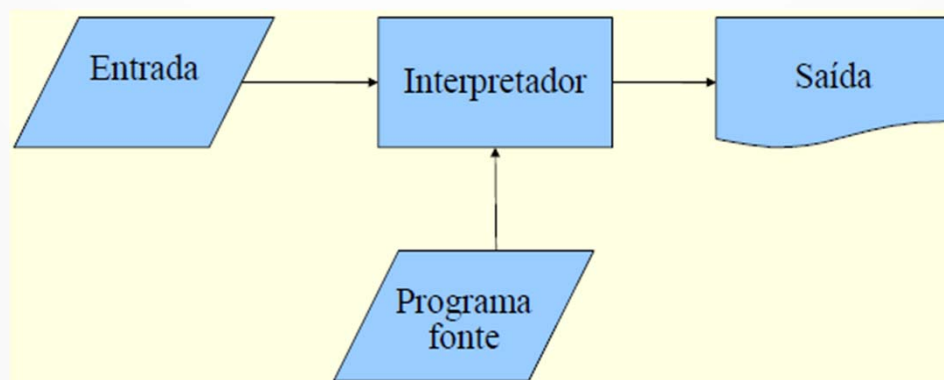
- O **programa-objeto** nem sempre está pronto para ser executado.
 - Outros trechos de código precisam ser incluídos.
 - Bibliotecas
 - Subprogramas
- Assim, deve-se montar o **programa executável** por meio de um programa chamado *link-editor* ou *linking-loader*.
 - Compilador normalmente chama o *link-editor* de forma automática.

Link-Editores



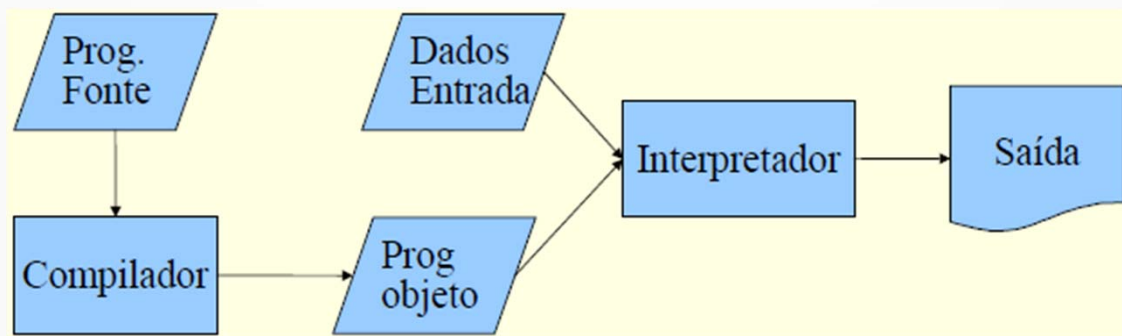
Interpretadores

- **Interpretadores** simulam uma máquina virtual (ou *virtual machine* - VM).
- Programa fonte é lido, entendido e as instruções são executadas imediatamente. Ex.: Python, PHP ...



Esquemas Híbridos

- Compilador gera código para uma máquina virtual (bytecodes).
- Máquina virtual é executada separadamente lendo bytecodes e interpretando-o. Ex.: Java



Por que Python?

- Simples o suficiente para um curso introdutório.
- Além de simples, os códigos foram feitos para serem intuitivos.
- Muitos recursos:
 - Orientação a Objetos
 - Escalável (módulos, classes, controle de exceções)
 - Biblioteca embutida extensa e
 - Grande número de módulos fornecidos por terceiros
- Grande variedade de aplicações.
- Linguagem interpretada (script).
- Multi-plataforma.
- Gratuito.
- Comunidade bastante desenvolvida.

O que vamos precisar

- Uma implementação da linguagem
 - <https://www.python.org>
 - Implementação pronta para baixar (windows)
 - GNU/Linux normalmente já vem com Python instalado
- Um editor de textos (qualquer editor serve). Exemplos:
 - PyCharm Community
 - VS Code
 - Ambiente IDLE inclui um editor
 - Incluído na distribuição Windows
 - Pode ser instalado nos sistemas GNU/Linux

Python Interativo

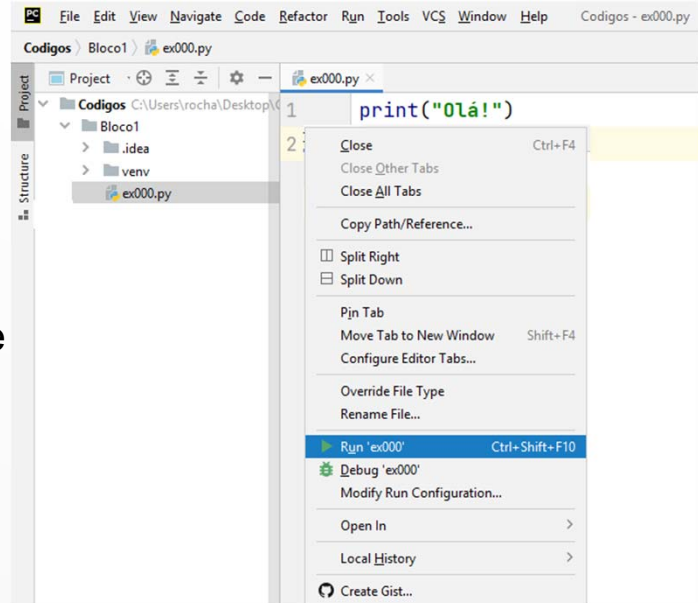
- Rode o interpretador IDLE
- Digite comandos python
- Cada comando é executado imediatamente

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41)
[MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more
information.
```

```
>>> print("Olá!")
Olá!
>>>
```

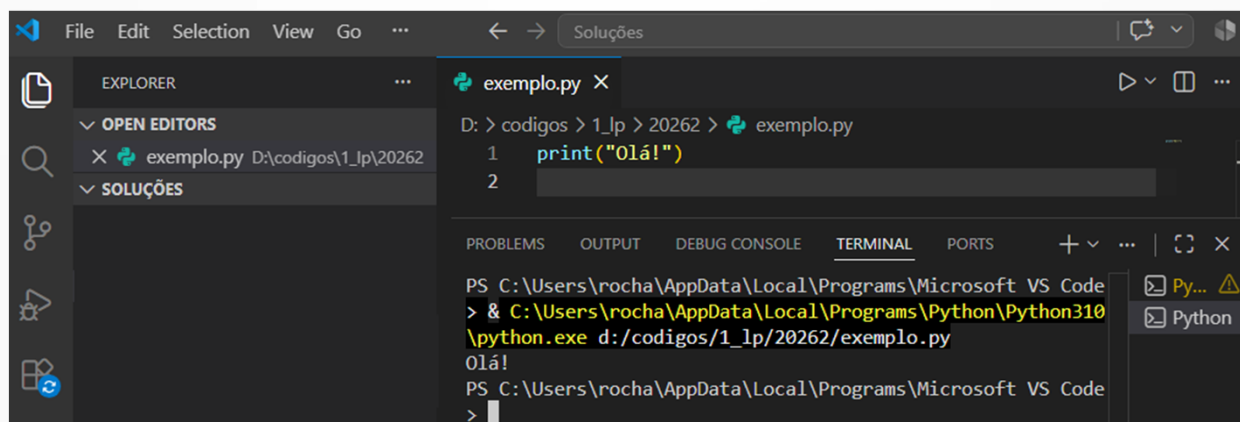
Executando um programa Python

- Interfaces gráficas normalmente já associam os sufixos **.py** e **.pyc** com o interpretador.
- No Jupyter a extensão é **.ipynb**
- Ao lado, o exemplo no IDE PyCharm:



Executando um programa Python

- Abaixo, um exemplo no IDE VS Code:



Programas: observações importantes

- **Programas** são códigos (sequência de instruções) que dizem ao computador o que fazer.
- Se você não diz (escrever) o que fazer, o computador não faz.
- Se você diz (escrever), o computador vai fazer exatamente o que você disse (Ex.: algoritmo do rótulo do xampu).
- Escreva uma instrução por linha. (cada linha representa algo).
- Exemplo:

```
preco = 20.00
quantidade = 4.00
valor_total = preco * quantidade      # 80.00
```
- As instruções (as linhas de códigos) são lidas/executadas de cima para baixo. No exemplo acima, como saber o valor total ?
- Quando erros ocorrem a execução é interrompida e lhe avisa o que aconteceu.

Regras de destaque

- Errar é normal, faz parte do processo de aprendizado.
- Não se frustre com o erro, vamos achá-lo e resolvê-lo.
- Não decore código, apenas codifique.
- Bom mesmo é se divertir, tente!! Você aprende mais!! É legal!!
- Procurar “as coisas” por conta própria e testar vai ajudar muito.
- Lembre-se, os professores e outros alunos estão aqui para lhe ajudar.